

Few-Shot Prompting

Day 2 — Notebook 2 of 5 | Estimated Time: 25 minutes

What You'll Learn

- What few-shot prompting is and when it outperforms zero-shot
- How to structure examples for consistent outputs
- How to enforce output formats (JSON, markdown tables) using examples

What is Few-Shot Prompting?

In **few-shot prompting**, you provide the model with **a few examples** of the desired input-output behavior before giving it the actual task. The model learns the pattern from your examples and applies it.

```
Zero-shot: "Classify as positive/negative: 'I love this product'"
Few-shot:  "Examples:
            'Great quality!' → Positive
            'Terrible service' → Negative
            'It works fine' → Neutral

            Now classify: 'I love this product'"
```

When to use few-shot vs zero-shot:

- **Zero-shot:** Simple, well-understood tasks
- **Few-shot:** When you need specific output formats, domain-specific patterns, or the model struggles with zero-shot

Setup

```
%pip install -U -q "google-genai>=1.0.0"
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->goog
```

```
import sys, os
import time

import json
import re
from google import genai
from google.genai import types
from IPython.display import Markdown

try:
    from google.colab import auth
    auth.authenticate_user()
    print("✅ Authenticated via Colab")
except ImportError:
    print("✅ Using local credentials (gcloud auth)")

client = genai.Client(
    vertexai=True,
    project="chanakya-476010",
    location="us-central1",
)

MODEL_ID = "gemini-2.5-flash"
```

```
print("✅ Ready!")
```

✅ Authenticated via Colab
✅ Ready!

1. Basic Few-Shot: Sentiment Classification

Let's compare zero-shot vs few-shot on the same task.

```
# Zero-shot - sometimes gives verbose output
zero_shot_prompt = """
Classify the sentiment: "The delivery was late but the product quality was excellent."
"""

response = client.models.generate_content(model=MODEL_ID, contents=zero_shot_prompt)
print("Zero-shot result:")
print(response.text)
```

Zero-shot result:
The sentiment is **Positive**.

While there was a negative aspect (late delivery), the strong positive statement about the "excellent" product quality overrides

```
# Few-shot - examples teach the model the desired format
few_shot_prompt = """
Classify the sentiment of each review. Respond with only one word: Positive, Negative, or Mixed.

Review: "I love this product, it works perfectly!"
Sentiment: Positive

Review: "Worst purchase ever. Complete waste of money."
Sentiment: Negative

Review: "It works, but the instructions were confusing."
Sentiment: Mixed

Review: "The delivery was late but the product quality was excellent."
Sentiment:
"""

response = client.models.generate_content(model=MODEL_ID, contents=few_shot_prompt)
print("Few-shot result:")
print(response.text.strip())
```

Few-shot result:
Mixed

💡 **Notice:** Few-shot prompting gives a much more **consistent and concise** output because the model learns the pattern from the examples.

2. Enforcing JSON Output with Few-Shot

One of the most powerful uses of few-shot prompting is enforcing structured output formats.

```
prompt = """
Extract product information and return as JSON.

Input: "The iPhone 15 Pro by Apple costs $999 and has 256GB storage."
Output: {"product": "iPhone 15 Pro", "brand": "Apple", "price": 999, "storage": "256GB"}

Input: "Samsung Galaxy S24 Ultra is priced at $1,299 with 512GB."
Output: {"product": "Galaxy S24 Ultra", "brand": "Samsung", "price": 1299, "storage": "512GB"}

Input: "Google Pixel 9 Pro costs $799 and comes with 128GB storage."
Output:
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(response.text.strip())
```

```
```json
{"product": "Pixel 9 Pro", "brand": "Google", "price": 799, "storage": "128GB"}
```
```

```
# Verify it's valid JSON
import json
```

```
try:
    parsed = json.loads(response.text.strip())
    print("✅ Valid JSON!")
    print(json.dumps(parsed, indent=2))
except json.JSONDecodeError as e:
    print(f"❌ Invalid JSON: {e}")
    print("Raw output:", response.text.strip())
```

```
❌ Invalid JSON: Expecting value: line 1 column 1 (char 0)
Raw output: ```json
{"product": "Pixel 9 Pro", "brand": "Google", "price": 799, "storage": "128GB"}
```
```

### 3. Few-Shot for Custom Classification

Few-shot is particularly useful when you need domain-specific categories.

```
Custom intent classification for an IT helpdesk
prompt = """
Classify the IT support ticket into one of these categories:
Hardware, Software, Network, Access, Other
```

```
Ticket: "My laptop screen is flickering and has dead pixels."
Category: Hardware
```

```
Ticket: "I can't connect to the VPN from home."
Category: Network
```

```
Ticket: "Excel keeps crashing when I open large files."
Category: Software
```

```
Ticket: "I need access to the marketing SharePoint site."
Category: Access
```

```
Ticket: "My keyboard is not working after the coffee spill."
Category:
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(f"Category: {response.text.strip()}")
```

```
Category: Hardware
```

### 4. Few-Shot for Format Transformation

Teach the model to transform data from one format to another.

```
Natural language to SQL
prompt = """
Convert the natural language query to SQL. The database has tables:
- employees (id, name, department, salary, hire_date)
- departments (id, name, budget)
```

```
Query: "Show all employees in the Engineering department"
SQL: SELECT * FROM employees WHERE department = 'Engineering';
```

```
Query: "What is the average salary by department?"
SQL: SELECT department, AVG(salary) as avg_salary FROM employees GROUP BY department;
```

```
Query: "Find employees hired after January 2024 with salary above 80000"
SQL: SELECT * FROM employees WHERE hire_date > '2024-01-01' AND salary > 80000;
```

```
Query: "List the top 5 highest paid employees with their department names"
SQL:
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(response.text.strip())
```

```
```sql
SELECT e.name, e.salary, d.name AS department_name
FROM employees AS e
JOIN departments AS d ON e.department = d.name
ORDER BY e.salary DESC
LIMIT 5;
```
```

## 5. Best Practices for Few-Shot Examples

| Practice                        | Why                                                        |
|---------------------------------|------------------------------------------------------------|
| Use <b>3-5 examples</b>         | Enough to establish pattern, not too many to waste tokens  |
| Make examples <b>diverse</b>    | Cover edge cases and variations                            |
| Keep examples <b>consistent</b> | Same format in every example                               |
| Order examples <b>logically</b> | Simple → complex, or randomized                            |
| Be <b>exact</b> in format       | If you want JSON with specific keys, show those exact keys |

### 🏆 Exercise 1: Build a Product Review Analyzer

Create a few-shot prompt that analyzes product reviews and returns JSON with: sentiment, key\_pros (list), key\_cons (list), and rating (1-5).

```
Exercise 1: Product review analyzer
TODO: Write a few-shot prompt with 2-3 examples, then test it

prompt = """

Review: "The headphones have amazing sound quality and are very comfortable,
but the battery life is disappointing and they're overpriced for what you get."

Analysis : "[sentiment: Neutral , key_pros: [sound quality, comfortable], key_cons: [battery life, overpriced], rating: 3.5]"

Review: "The airpods have excellent sound quality and are great for long hours of listening but they are 30% overpriced"

Analysis : "[sentiment: Neutral , key_pros: [sound quality, great for long hours of listening], key_cons: [overpriced], rating:

Review: "The Mac Laptop has all the latest features and gives a tablet for free , but the tablet has only 6 months warranty and

Analysis : "[sentiment: Semi Positive, key_pros: [latest features, tablet for free], key_cons: [only 6 months warranty] , rating:

Review: "The selfie stick is of great value and quality and probably the third best brand in the world"

Analysis:
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(response.text)

"[sentiment: Positive, key_pros: [great value, quality, third best brand in the world], key_cons: [], rating: 4.5]"
```

### 🏆 Exercise 2: Email Subject Line Generator

Create a few-shot prompt that generates professional email subject lines from email body summaries.

```
Exercise 2: Email subject line generator
TODO: Write a few-shot prompt with examples of body → subject line

test_bodies = [
 "Requesting approval for the Q3 marketing budget of $50,000 for digital advertising campaigns.",
 "Informing the team that the office will be closed on Friday for building maintenance.",
 "Following up on the client meeting from yesterday and sharing the action items.",
```

```
]

for body in test_bodies:
 prompt = f"""
 Create a subject line for an email which is action oriented. There should be only one statement not more than that Here are

 Body : There is a holiday on Tuesday
 Subject : Holiday Notification

 Body : Preponement of exams from Tuesday to Monday
 Subject : Exams preponement

 Body : Summer Vacation from 16th May 2026 to 26th May 2026
 Subject : Summer Vacation Notification

 Body: "{body}"
 Subject:
 """

 response = client.models.generate_content(model=MODEL_ID, contents=prompt)
 print(f"Body: {body[:60]}...")
 print(f"Subject: {response.text.strip()}")
 print()
```

Body: Requesting approval for the Q3 marketing budget of \$50,000 f...  
Subject: Q3 Marketing Budget Approval Request

Body: Informing the team that the office will be closed on Friday ...  
Subject: Office Closure

Body: Following up on the client meeting from yesterday and sharin...  
Subject: Client Meeting Action Items

 Further Reading

| Resource                                            | Type  | Link                                                                                                                        |
|-----------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------|
| Few-Shot Prompting                                  | Blog  | <a href="https://promptingguide.ai/techniques/fewshot">promptingguide.ai/techniques/fewshot</a>                             |
| Language Models are Few-Shot Learners (GPT-3 Paper) | Paper | <a href="https://arxiv.org/abs/2005.14165">arxiv.org/abs/2005.14165</a>                                                     |
| Google Prompting Strategies                         | Docs  | <a href="https://ai.google.dev/gemini-api/docs/prompting-strategies">ai.google.dev/gemini-api/docs/prompting-strategies</a> |

Next up: [03\\_Chain\\_of\\_Thought\\_Prompting.ipynb](#) — Make the model show its reasoning